

RELAZIONE HOMEWORK 1 DSBD

Luca Eterno – Rosario Grasso

I. Abstract

Il progetto implementa un sistema distribuito basato su microservizi per:

- la **gestione degli utenti** (registrazione, cancellazione, verifica credenziali);
- la **gestione degli aeroporti di interesse** associati a ciascun utente;
- la raccolta e la persistenza di **dati di volo** (arrivi/partenze) tramite integrazione con le API di **OpenSky Network**.

L'architettura è composta da due microservizi principali:

1. **User Manager**, responsabile di utenti e credenziali;
2. **Data Collector**, responsabile di aeroporti, associazioni utente–aeroporto e dati di volo.

I microservizi comunicano tra loro tramite **gRPC** (User Manager ↔ Data Collector) per verificare l'esistenza e la validità delle credenziali utente.

I dati sono memorizzati in **due database MySQL distinti**, ciascuno eseguito in un Docker Container separato. L'intero sistema viene orchestrato tramite **Docker Compose**, che avvia:

- due container MySQL (user-manager-db, data-collector-db);
- due container per i microservizi: (user_manager, data_collector);
- un container redis: per l'implementazione della politica at-most-once.

II. Schema Architetture

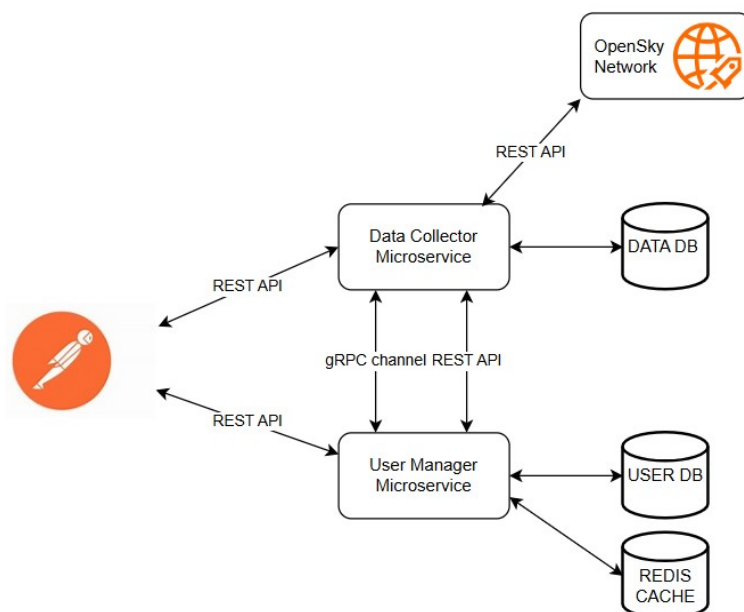


Figure 1: Diagramma architetturale

Il diagramma architetturale mostrato in figura 1 mostra:

- un **Client esterno** (Postman) che invia richieste REST:
 - a **User Manager** (porta 5003);
 - a **Data Collector** (porta 5002).
- **User Manager microservice:**
 - Interagisce con il database `usermanager_db` per verificare la presenza di un utente o la validità delle credenziali.
 - Espone il servizio gRPC con le RPC:
 - `CheckUserExists(email)`
 - `CheckUserCredentials(email, password)`
 - Espone API REST per:
 - registrare un nuovo utente;
 - cancellare un utente (e relativi aeroporti di interesse in Data Collector).
- **Data Collector microservice:**
 - Espone API REST per:
 - gestire le associazioni utente–aeroporto;
 - interrogare i dati di volo (ultimo volo, statistiche);
 - richiamare OpenSky per aggiornare i dati dei voli.
 - Comunica con:
 - il database `data_db`;
 - lo User Manager gRPC per verificare esistenza utente e credenziali;
 - le API pubbliche di **OpenSky** per recuperare i voli di arrivo/partenza per un certo aeroporto, utilizzando autenticazione tramite token OAuth2 (Client Credentials + Bearer Token) con caching.
- **User DB** (`usermanager_db`): contiene la tabella `users` del dominio User Manager.
- **Data DB** (`data_db`): contiene le tabelle `user_airports`, `flights` del dominio Data Collector.
- **Redis** (`redis_db`): DB in-memory in un container separato, usato come cache di `requestID`, contattato da User Manager per verificare l'unicità delle richieste per la politica at-most-once. La chiave contiene: tipo di richiesta (`request` | `delete`), `requestID`, hash del body.
- **OpenSky Network** (API REST): microservizio esterno per la raccolta dei dati di volo.

III. Interazioni tra i componenti e API implementate

Questa sezione descrive nel dettaglio i flussi di comunicazione:

Client → User Manager

Il client (Postman) invia richieste HTTP verso `user-manager (app.py)`:

- **register** - POST `/users/register` → body JSON (`email, cf, password`):
 - Verifica i campi della request e gestisce eventuali eccezioni;
 - Verifica l'idempotenza della richiesta (per richieste duplicate) (at-most-once);
 - Registra un nuovo utente nel DB `usermanager_db`.
- **delete_user** via DELETE `/users/delete` → body JSON (`email, password`):
 - Verifica i campi della requeste e gestisce eventuali eccezioni;
 - Verifica l'idempotenza della richiesta (per richieste duplicate) (at-most-once);
 - Verifica l'esistenza e le credenziali dell'utente;
 - Cancella l'utente e i relativi interessi.

Il client riceve in risposta un JSON contenente dettagli sull'operazione e status code.

User Manager → User DB

User Manager interagisce con il database `usermanager_db` per tutte le operazioni legate al ciclo di vita dell'utente:

- **Create:** POST `/users/register`:
 - inserisce una nuova riga nella tabella `users` con `email, cf, password`.
- **Delete:** DELETE `/users/delete`:
 - cancella l'utente dalla tabella `users` e relativi interessi in `user_airports`.

User Manager → Redis

User Manager interagisce con il database `redis_cache` per l'attuazione della politica di at-most-once.

User Manager → Data Collector (cancellazione interessi)

Dopo una cancellazione utente avvenuta con successo:

- User Manager invia una richiesta REST verso Data Collector:
 - DELETE `/user/airports` → body JSON `{"email": email_deleted_user}`.
- Data Collector elimina tutte le righe della tabella `user_airports` associate a quell'email.
- Questo meccanismo garantisce la **coerenza dei dati**: un utente cancellato non ha più aeroporti associati.

Client → Data Collector

Il client (Postman) interagisce con Data Collector tramite REST per:

- **Gestione aeroporti di interesse dell'utente:**
 - **add_user_airport** - POST /user/airports per associare un utente ad un aeroporto → body JSON (email, password, airport_code):
 - Verifica i campi della request e gestisce eventuali eccezioni;
 - Verifica le credenziali dell'utente via gRPC su User Manager;
 - Aggiungo l'interesse dell'utente per l'aeroporto.
 - **get_user_airports** - GET /user/airports?email=... per ottenere l'elenco degli aeroporti associati ad un utente;
 - Verifica i campi della request e gestisce eventuali eccezioni;
 - Verifica l'esistenza dell'utente via gRPC su User Manager;
 - Restituisce la lista degli aeroporti associati ad un utente.
 - **delete_user_airports** - DELETE /user/airports per rimuovere una singola associazione o tutte le associazioni di un utente → body JSON (email, password, airport_code(opzionale)).
 - Verifica i campi della request e gestisce eventuali eccezioni;
 - Verifica le credenziali dell'utente via gRPC su User Manager;
 - Rimuove una singola associazione o tutte le associazioni di un utente.
- **Gestione dati di volo:**
 - **refresh_flights_for_airport** - POST /airport/<airport>/refresh-flights per chiamare OpenSky e aggiornare i voli in DB → body JSON (hours (24h se assente), direction (both se assente))
 - Verifica i campi della request e gestisce eventuali eccezioni;
 - Chiama **refresh_flights_for_airport_logic** per impostare il body della richiesta http a opensky;
 - **fetch_flights_from_opensky** per accedere ai dati di volo offerti dalla API di opensky;
 - **store_flights_in_db** per archiviare i dati di volo.
 - **get_user_flights** - GET /user/flights/<email> per ottenere la lista dei voli associati agli aeroporti d'interesse:
 - Verifica i campi della request e gestisce eventuali eccezioni;
 - Verifica l'esistenza dell'utente via gRPC su User Manager;
 - Interroga il database e restituisce la lista dei voli associati agli aeroporti d'interesse.

- **get_last_flight_for_user_airport** - GET /user/flights/last per ottenere l'ultimo volo di arrivo/partenza per l'aeroporto indicato:
 - Verifica i campi della request e gestisce eventuali eccezioni;
 - Verifica l'esistenza dell'utente via gRPC su User Manager;
 - Verifica che l'aeroporto sia associato all'utente;
 - Interroga il database e restituisce l'ultimo volo di arrivo/partenza per l'aeroporto indicato.
- **get_average_flights_for_user_airport** - GET /user/flights/average calcola la media dei voli in arrivo/partenza negli ultimi X giorni per l'aeroporto indicato:
 - Verifica i campi della request e gestisce eventuali eccezioni;
 - Verifica l'esistenza dell'utente via gRPC su User Manager;
 - Verifica che l'aeroporto sia associato all'utente;
 - Interroga il database e restituisce la media dei voli in arrivo/partenza negli ultimi X giorni per l'aeroporto indicato.

Il client riceve in risposta un JSON contenente dettagli sull'operazione e status code.

Data Collector → User Manager (via gRPC)

Per garantire che ogni operazione sugli aeroporti sia eseguita solo per utenti validi, Data Collector utilizza il microservizio User Manager gRPC:

- **CheckUserExists(email)**, viene invocato per verificare che l'utente esista.
- **CheckUserCredentials(email, password)**, viene invocato per verificare le credenziali dell'utente.

Abbiamo scelto di utilizzare un controllo più stringente (CheckUserCredentials) per le operazioni di POST/DELETE e un controllo meno stringente (CheckUserExists) per le operazioni di GET.

Data Collector → Data DB

Data Collector interagisce con il database data_db per due scopi principali:

- **Gestione associazioni utente-aeroporto** (tabella user_airports):
- **Gestione dati di volo** (tabella flights):

Data Collector → OpenSky Network

Data Collector effettua chiamate HTTP verso le API di OpenSky per raccogliere le informazioni sui voli degli aeroporti di interesse. Per avere pieno accesso alle API di OpenSky è necessaria una prima fase di autenticazione:

- **OAuth2 Client Credentials** → chiamata all'endpoint di token OpenSky;
- token **access_token** memorizzato in cache in memoria;

Una volta ottenuto il token, Data-collector effettua richieste HTTP alle API OpenSky in due modi:

1. Come da consegna, mediante un **thread** che effettua il refresh automatico ogni 12h dei voli di interesse non ancora presenti all'interno del database;
2. Mediante l'end point **refresh_flights**, aggiunto inizialmente come end point di test, abbiamo deciso di mantenerlo per eventuali sviluppi futuri che permettano il refresh della lista dei voli anche all'aggiunta di un nuovo aeroporto di interesse.

IV. Generalità sulla soluzione proposta e conclusioni

Il progetto sviluppato soddisfa pienamente i requisiti definiti nella consegna, realizzando un sistema distribuito basato su microservizi containerizzati, orchestrati tramite Docker Compose, e articolato in più componenti che cooperano mediante API REST e gRPC. In particolare:

- Gestione della registrazione e cancellazione utenti, tramite API REST.
- Persistenza utenti in un database relazionale containerizzato.
- Implementazione della politica “**at-most-once**” sulle operazioni di registrazione, grazie all'utilizzo di **Redis** come archivio degli identificativi delle richieste.
- Gestione degli interessi dagli utenti tramite API REST.
- Verifica dell'esistenza dell'utente tramite canale gRPC verso User Manager.
- Recupero dati voli da OpenSky Network tramite API ufficiali REST (con autenticazione tramite token).
- Implementazione delle funzionalità richieste: recupero dell'ultimo volo in arrivo/partenza e calcolo della media dei voli negli ultimi X giorni.
- Tutti i servizi (User Manager, Data Collector, User DB, Data DB, Redis) sono contenuti in container separati.

Motivazioni principali del modello architetturale:

- Separazione netta tra i domini funzionali (*user management vs data management*).
- Riduzione delle dipendenze dirette grazie all'uso del gRPC point-to-point.
- Containerizzazione totale per un deployment ripetibile, isolato e portabile.
- Architettura facilmente estensibile.

Per garantire coerenza dei dati e un funzionamento robusto, sono state introdotte alcune funzionalità oltre i requisiti minimi:

- Cancellazione automatica degli aeroporti associati all'utente
- Endpoint dedicato per eliminare gli interessi utente

Uno degli aspetti più delicati della consegna è l'implementazione della policy “**at-most-once**”, cioè garantire che un'operazione (in particolare la registrazione utente) non venga eseguita più di una volta, anche in presenza di ritrasmissioni accidentali da parte del client, richieste duplicate e ritardi di rete o timeout.

Per questo motivo è stato scelto di utilizzare Redis, un database in-memory estremamente veloce, usato per:

1. **Registrare ogni richiesta** REST ricevuta (attraverso la tupla tipo di richiesta + requestID + hash del body);
2. **Verificare se la richiesta è presente in memoria** prima di eseguire l'operazione.
3. **Restituire la stessa risposta** precedentemente memorizzata, in caso di duplicazione.